

Introduzione ai Laboratori di Machine Learning

Dr. Rosario Leonardi

rosario.leonardi@unict.it

Prof. Giovanni Maria Farinella

giovanni.farinella@unict.it

www.dmi.unict.it/farinella/

Laboratori di Machine Learning

- **Formato dei Laboratori:** esercitazione pratica + domande. Esercizi per casa
- **Portare il proprio laptop a lezione** (è possibile seguire in gruppi di 2/3 se necessario)
- **Materiale didattico disponibile sul sito/team del corso**

Progetto

- **Modalità:** Lavoro in gruppi da 2–3 persone
- **Valutazione:** Il progetto pesa per il 50% sul voto finale
- **Bonus presenze:** +2 punti sul voto del laboratorio per chi raggiunge almeno l'80% di presenze in laboratorio
- **Template GitHub:** Utilizzare il repository template come punto di partenza per l'organizzazione del codice, dati, notebook e documentazione
- <https://github.com/leonardirosario97/unict-ml-year-x-group-x>



Linguaggio di programmazione ad **alto livello**, **interpretato** e **multi-paradigma**, ampiamente utilizzato nel calcolo scientifico e nell'AI.

Caratteristiche principali:

- **Sintassi chiara** (leggibilità e rapidità di sviluppo)
- **Ampio ecosistema** di librerie scientifiche e di machine learning
- **Multi-piattaforma e portabile**
- **Interattivo** (supporto nativo a Jupyter Notebook)
- **Grande community** e documentazione estesa

Stack SciPy

- **NumPy**: Supporto per array e matrici multidimensionali
- **SciPy**: Algoritmi per il calcolo scientifico
- **Matplotlib**: Creazione di grafici e visualizzazione dati 2D/3D
- **IPython / Jupyter**: Shell e ambiente interattivo per la prototipazione
- **Pandas**: Manipolazione e analisi di dati tabulari (DataFrame)
- **SymPy**: Calcolo algebrico e matematico simbolico

NumPy 

SciPy 

matplotlib 

IP[y]: IPython
Interactive Computing


SymPy

 **pandas**

Stack SciPy + PyTorch

- **NumPy**: Supporto per array e matrici multidimensionali
- **SciPy**: Algoritmi per il calcolo scientifico
- **Matplotlib**: Creazione di grafici e visualizzazione dati 2D/3D
- **IPython / Jupyter**: Shell e ambiente interattivo per la prototipazione
- **Pandas**: Manipolazione e analisi di dati tabulari (DataFrame)
- **SymPy**: Calcolo algebrico e matematico simbolico
- **PyTorch**: Framework di Deep Learning per il calcolo su tensori e GPU, differenziazione automatica (autograd) e addestramento di reti neurali



<https://pytorch.org/>

Versioni di Python

- **Python 3:** Lo standard di riferimento odierno del linguaggio (rilasciato nel 2008, include il supporto nativo a Unicode, type hinting e performance ottimizzate).
- **Python 2:** *End-of-Life* ufficiale dal 1° gennaio 2020 (completamente dismesso e non più supportato dalle moderne librerie scientifiche).

In questo corso utilizzeremo Python 3.10

Distribuzioni di Python

Il modo più semplice per installare tutto l'occorrente è utilizzare una **distribuzione Python**.

Esistono diverse distribuzioni (Anaconda, Miniconda, Micromamba).
Per il corso utilizzeremo l'ecosistema **Conda / Anaconda**.



Windows



macOS



Linux

<https://www.anaconda.com/download/>

Jupyter Notebook



- **Jupyter Notebook** è un ambiente interattivo che consente di scrivere ed eseguire codice Python direttamente all'interno di celle, ottenendo risultati immediati.
- Permette di combinare codice, testo formattato (con Markdown e LaTeX), grafici e visualizzazioni interattive in un unico documento.
- Dataset pesanti e modelli restano caricati in RAM/GPU durante la prototipazione.
- È ampiamente utilizzato in ambito scientifico, didattico e di ricerca per analisi dati, machine learning e prototipazione veloce.
- I notebook possono essere facilmente condivisi, favorendo la riproducibilità e la collaborazione.

Installazione di PyTorch

Dopo aver installato Anaconda, seguire le istruzioni disponibili al seguente link per installare PyTorch:

<https://pytorch.org/get-started/locally/>

